# New CP October Demo Planning

*Meeting guide for 4 October 2026*

Meeting lead: Aya Adel | Suggested duration: 40 minutes

### Opening and current position

We want to agree on a realistic October demo scope and give everyone a clear preparation task for tomorrow. The demo should show a complete business scenario working against the new database, with validated data and visible results.

- Frontend selection is pending. The backend direction is Node.js with Express or NestJS.
- The database design is close to completion. We will review InsightHub and existing components for reuse.
- Planning, migration validation, and initial implementation should run in parallel, with an early database-read proof.

### Proposed demo scope

Plan A is the proposed delivery baseline for Models and Conversion services. It demonstrates one agreed representative service option from each category within existing orders, using APIs connected to the new PostgreSQL database. Plan B adds features only after Plan A is stable and remaining capacity is confirmed.

| Area | Plan A minimum scope | Plan B expanded scope |
|---|---|---|
| Access | Google login and basic application permissions for viewing reports, adding/editing services, and performing the selected transition. | Additional roles and permissions. |
| Reporting | One combined Models and Conversion report with service-type/status filters, search, simple counts, and navigation to order/service details. | Additional reports and export. |
| Service changes | Add and edit one agreed Models option and one agreed Conversion option within existing orders; validate the selected mandatory fields and business rules. | Create a new order containing the selected Models and/or Conversion options. |
| Workflow | One agreed permitted status transition for each selected service, with actor, timestamp, and previous/new status recorded. | Additional workflow steps, assignments, and due dates. |
| Files and email | Display existing attachments linked to the selected demo records. | S3 upload and retrieval plus an event email using an editable template. |
| Operational monitoring | Prometheus scrape verification and Grafana API traffic/status/latency panels with recorded demo observations. | Controlled test-only failure/recovery and tested alerts if already ready. |

### Proposed end-to-end demo scenario

#### Business story and demo objective

An authorized CP clerk handles two existing customer orders: one requiring the selected Models service and one requiring the selected Conversion service. The clerk checks the existing information, adds each service, corrects an

editable detail, advances it through one permitted workflow transition, and verifies the resulting report and history. The presenter then shows how the API activity appears in Prometheus and Grafana.

The demo proves three outcomes: correct business behavior, persistence in PostgreSQL, and visibility of application health and API activity during the same scenario. Monitoring figures will be measured during rehearsal and the live demo; this guide contains expected observations, not completed test results.

### Preparation and agreed business rules

- Use two prepared existing orders, labeled Demo-M and Demo-C for presentation. Use one order for both services only if confirmed by the business rules. Prepare appropriate existing attachments and reference values.
- For each service, confirm the supported option, required fields, editable fields, applicable duplicate rules, initial status, allowed next status, and permitted actions. “Initial status” and “next status” below are placeholders for agreed business statuses.
- Prepare a clerk account with the agreed permissions. An additional restricted account is optional for showing access control if already available.
- Record baseline service counts, statuses, order relationships, and relevant field values. Validate the selected migrated records against the legacy source; the migration team owns execution and corrections.
- Backend/developer: expose application metrics, configure the Prometheus scrape target, connect Grafana to Prometheus, and prepare a dashboard filtered to the demo environment and API job. Confirm metrics from an actual test request before rehearsal.
- Use one agreed time window and timezone across the demo and dashboard; mark the start and end. Capture a baseline after a successful scrape and allow for the configured scrape/dashboard refresh intervals.
- Use normalized route labels and bounded values such as method and status code. Keep order identifiers, customer details, patient details, and free text out of metric labels.

### Live demonstration: actions and visible results

| Step / suggested time | Presenter action | Expected business / data result | Monitoring evidence |
|---|---|---|---|
| 1. Baseline (2 min) | Open Grafana, select the demo environment/time window, and show the Prometheus target before starting the business flow. | Confirm the prepared records and baseline counts are ready. | Target is UP; recent samples are available. Show starting API counters, latency panels, and runtime memory if instrumented. UP confirms scrape reachability; verify application readiness separately. |
| 2. Sign in and orient (2 min) | Sign in through Google; open the combined Models/Conversion report and explain search, filters, counts, and order navigation. | Clerk sees only the agreed actions; report values match the prepared baseline. | Report API calls appear in the selected job/route metrics after collection. Authentication calls appear only where the application instruments them. |
| 3. Inspect Models order (2 min) | Search for Demo-M; open its order/customer context, existing services, and attachment references. | Order relationships and displayed values match the prepared PostgreSQL record. | Read/detail request counters increase; latency is available for the instrumented routes. |

| | | | |
|---|---|---|---|
| 4. Validate then add Models (3 min) | Choose the agreed Models option. Leave one required field empty, attempt save, explain the message, then enter valid data and save. | Invalid input creates no service; valid input creates one service linked to Demo-M with the agreed initial status. | Successful write is reflected in API metrics. A client-side rejection generates no API call; a server-side rejection is counted as an expected 4xx response. |
| 5. Correct and reopen Models (2 min) | Change an agreed editable field, save, leave the page, and reopen the service. | New value persists in PostgreSQL; order link and unchanged fields remain correct. Show the recorded change history. | Write/read activity and response times appear for the relevant API routes. |
| 6. Advance Models (2 min) | Show the permitted next action, perform it, then reopen history. If already supported, attempt one prohibited transition. | Allowed transition persists with actor, timestamp, and previous/new status. A prohibited action leaves the status unchanged. | Successful transition request is visible. If the server rejects a prohibited action, show the expected 4xx separately from 5xx failures. |
| 7. Reconcile Models report (2 min) | Return to the report; filter by Models and the new status; locate the same service. | Exactly one intended Models service was added. Status-filter counts reflect the transition; editing or transitioning did not create another service. | Report refresh generates observable read requests; compare with the baseline without equating API request count to service count. |
| 8. Complete Conversion flow (5 min) | Find Demo-C; show its context and attachments. Add the agreed Conversion option, validate required input, edit and reopen, then perform its permitted transition. | The same persistence/history checks pass using Conversion-specific rules. One intended Conversion service is added and remains linked to Demo-C. | Show the corresponding API activity. If both categories share routes, distinguish their records in the application; do not claim separate service metrics unless instrumented. |
| 9. Combined business result (2 min) | Show both categories in the report, then open each final record/history. Compare with the recorded baseline. | For this prepared run, total service count increases by two; report scope/filter counts agree with the actual dataset. Both final values and statuses are correct. | Separate measured API activity from database-derived business counts; retries and background reads can cause additional requests. |
| 10. Operational results (4 min) | Return to Grafana with the same time window; show target status, requests/statuses, average and p95 API duration, and process memory if available. Open Prometheus to verify one metric behind a panel. | Explain whether any unexpected failure affected the completed business scenario. | Requests increased during execution; expected 4xx validations are identified; report actual 5xx values and measured latency. Relate graph activity to demo timing after scrape/refresh delays. |

| | | | |
|---|---|---|---|
| 11. Close with evidence (2 min) | Present the result sheet, screenshots, known limitations, and supported scope. | Confirm pass/fail for Models and Conversion and the demonstrated data relationships. | Record observed results and any gaps. A short demo demonstrates instrumentation and sample behavior; it does not establish production capacity. |

### Prometheus and Grafana: required demo outputs

Prometheus collects the instrumented metrics; Grafana displays them over the selected time window. Use request counters for traffic/status outcomes and a duration histogram for average/p95 latency. Metric names depend on the implemented exporter or API instrumentation; missing instrumentation must be recorded as a gap.

| Panel / result | Expected observation | Actual result to record |
|---|---|---|
| Prometheus target and sample freshness | Demo API scrape target is UP and samples are recent. Pair this with an application readiness check. | Target/job: ____; last sample/scrape: ____; readiness result: ____ |
| API requests by route/method | Activity rises for report, detail, create, edit, and transition routes during the demo. Counter differences or windowed increases are monitoring evidence, not exact business transaction counts. | Observed activity / time window: ____ |
| Response outcomes | Separate successful responses, expected 4xx validation/access rejections, and unexpected 5xx failures. Proposed pass condition: zero unexpected 5xx for the selected demo routes/window. | 2xx: ____; expected 4xx: ____; unexpected 5xx: ____ |
| Mean and p95 API response duration | Show measured seconds or milliseconds per selected route. Agree on a demo threshold after rehearsal; report sample size and window. Sparse samples make p95 estimates weak. | Window/sample count: ____; mean: ____; p95: ____; agreed threshold: ____ |
| API process memory (if instrumented) | Show the process memory trace across the demo; investigate a visible unexplained rise. A brief run cannot confirm absence of leaks. | Metric/unit: ____; start: ____; end: ____; observation: ____ |
| Dashboard evidence | Save a dashboard screenshot or link with the selected environment, time range, units, filters, and demo start/end. | Dashboard/evidence reference: ____ |

### Controlled failure demonstration — optional Plan B

If already available, use a test-only failure mechanism in the demo environment to generate one controlled failed request and show its 5xx metric, then run a successful request to show recovery. Mark and record this interval separately from the successful baseline scenario. Do not interrupt shared services to create the failure. Alerts may be shown only if their rules and timing have already been configured and tested.

### Acceptance and evidence checklist

- Both selected services complete add → edit → reopen → permitted transition → report/history, using the new PostgreSQL database.
- Invalid input is rejected without creating a partial service. Actual backend rejection behavior is verified where applicable.
- Values, statuses, and order links persist after reopening; each selected change is recorded in history. Transitions include actor, timestamp, and previous/new status.
- For the prepared run, the report reconciles with the two intended additions and the final statuses; no unintended extra service is created by edit/transition actions.
- Existing attachment references display correctly and the agreed application permissions apply.
- Prometheus collects fresh demo API metrics, and Grafana shows the same time window with route/status activity and measured latency. Required empty panels are treated as instrumentation gaps.
- Record actual 5xx results, latency measurements, and their sample/window context; compare latency against a threshold agreed after rehearsal.
- Capture final Models and Conversion records, histories, report counts, dashboard evidence, and a pass/fail result for each check.

### Responsibilities and rehearsal outputs

| Owner | Output |
|---|---|
| Frontend interns | Prepare report/detail/service forms, validation feedback, loading/error states, and the repeatable screen sequence. |
| Backend interns | Implement/verify agreed Models and Conversion rules, persistence, transitions/history, and API instrumentation with developer review. |
| Developer / monitoring owner | Connect and validate Prometheus/Grafana, prepare route/status/latency panels, record rehearsal measurements and any instrumentation gaps. |
| Aya Adel | Confirm selected service rules and expected results; validate business data, report reconciliation, and the presentation story. |
| Migration contact | Provide and correct the selected migrated dataset and confirm unresolved data issues. |

### Optional Plan B extension

After Plan A is stable, create a new order with the selected Models/Conversion options. Add export, additional workflow steps, S3 upload, event email, or the controlled monitoring failure demonstration only when individually ready and tested.

### Monitoring references

Prometheus — Histograms and summaries: https://prometheus.io/docs/practices/histograms/
Grafana — Prometheus query editor:
https://grafana.com/docs/grafana/latest/datasources/prometheus/query-editor/

## Engineering foundation

- Reuse suitable existing capabilities; connect SonarQube to CI/CD and test the business scenario.
- Demonstrate Prometheus collection and Grafana panels for target status, API traffic/status outcomes, measured response time, and process memory where available; capture actual results for the demo time window.
- Use RabbitMQ for background jobs where needed, with a scheduler for timed execution.

**Questions to discuss**

- Which representative Models and Conversion options, mandatory fields, edit rules, and permitted status transitions will we use?
- Which current internship and developer tasks contribute directly to this scenario?
- Who will be available after 15 October, and for how many hours?
- What is ready today, what needs adaptation, and which dependencies need a named owner?

### Delivery milestones

| October dates | Proposed outcome |
|---|---|
| 5 to 8 | Confirm the scenario, technology choices, responsibilities, and initial API contract. |
| 9 to 15 | Validate the required migrated data and deliver the first page reading from PostgreSQL. |
| 16 to 24 | Complete Plan A through incremental implementation, integration, and testing. |
| 25 to 28 | Stabilize the application, fix defects, and rehearse the complete demo. |
| 29 to 31 | Present the working scenario, evidence of progress, and the next priorities. |

### Preparation for 5 October

Task: October Demo Readiness and Scope Proposal. Prepare one short shared readiness sheet for the end-of-day meeting. Keep interns within their normal three-hour workday; use existing POCs and documentation as the starting point.

| Proposed role | Preparation and expected output |
|---|---|
| Frontend interns | Sketch the report and order or service screens. List reusable POC components and the data each screen needs. |
| Backend interns | Map the scenario to the new database and outline the required APIs. Record unclear rules or missing data dependencies. |
| Developer | Review reusable components and try a small database read using the intended application access. Provide evidence or the exact access blocker. |
| Aya Adel | Provide a representative case, expected behavior, and acceptance checks. Validate business meaning against the legacy system. |
| Migration contact | Confirm the available migrated dataset, known issues, and the owner for migration corrections. |

The migration team owns execution and corrections. Business validation confirms that the selected records preserve the expected relationships, values, and behavior.

### Review at the end of tomorrow

- Review the scenario proposal, reuse evidence, data readiness, and blockers.
- Agree on Plan A, suggested owners, immediate next steps, and the first implementation checkpoint.
- Set the review point for adding Plan B items once Plan A is stable.

### Record before closing today

Selected scenario or options: _____

Tomorrow task owners and review time: _____

Blocking dependencies and owners: _____

Technology decision owner and target date: _____

Demo success check: for both Models and Conversion, add and edit the selected service within an existing order, save it to PostgreSQL, reopen it to verify persistence, perform one permitted transition, and confirm the updated report and recorded history.